Student notes — Session 14 · Working with tables and matrices

Slides: [`Class_14_Tables_and_matrices.pdf`](../slides/Class_14_Tables_and_matrices.pdf). These notes are the notes pane of the student copy of the deck.

Slide 1 — Quiz 4: 25 minutes

This slide was on the screen during Quiz 4. Quiz 4 covered classes 11 and 12: characters, strings, string functions, and text-number conversions. Your marked paper comes back on Blackboard.

The **midterm is on Friday 30 October**. It covers classes 1 to 14, so this class is in it.

Slide 2 — Tables and matrices

The programs of this class are on Blackboard: Week 7 module, *Class 14 code*. It is a zip file. Unzip it and open the folder in Visual Studio Code. Every file has a comment at the top: what the program does, how to run it, and a list called **Try** with small changes to make. Make them.

How to run a program. Same as every week. In the terminal type `gcc -Wall name.c -o name` and press Enter, then `./name` (or `.\name` in Windows PowerShell) and press Enter.

Today: grids sent to functions, matrices, tables, and the cells around a cell. The last part is what Project 2 needs.

Slide 3 — What Tuesday gave us

What you need from Tuesday:

- `int t[2][3];` is a grid with 2 rows and 3 columns.
- `t[r][c]`: the row first, then the column. Both start at 0.
- `int t[2][3] = {{4, 7, 1}, {9, 2, 6}};` has one pair of braces per row.
- Two loops visit every cell: the outer loop for the rows, the inner loop for the columns.
- `#define ROWS 4` gives the size a name.

Slide 4 — Recap: what is on the screen?

Try to answer before you look at the next slide.

Slide 5 — Recap: what is on the screen?

The screen shows `row 0: 12` and `row 1: 17`. 4 + 7 + 1 = 12 and 9 + 2 + 6 = 17. `sum = 0` is inside the row loop, so every row starts a new sum. The file is `01_recap_rows.c`.

Slide 6 — Today's roadmap

Five parts:

1. **A grid goes to a function.**
2. **Matrices**: add, transpose, the diagonals.
3. **A table**: one answer per row or per column.
4. **The cells around a cell.**
5. **Your turn**: one function on paper.

Slide 7 — A grid goes to a function

Part 1: a grid as a parameter.

Slide 8 — The rule for a grid parameter

In the function header, write the grid like this:

```
void print_grid(int g[][COLS], int rows)
```

- The first brackets (the rows) can be empty.
- The second brackets (the columns) **must** have a number.
- The number of rows comes in as a normal parameter, `int rows`.

To call the function, write the name of the grid only, no brackets: `print_grid(a, 3);`

Slide 9 — A function that prints a grid

`print_grid` prints any grid with 4 columns. The loop for `r` goes up to `rows` (the parameter), and the loop for `c` goes up to `COLS`. `%3d` gives every number 3 places, so the columns line up. The file is `02_print_grid.c`.

Slide 10 — One function, two grids

`a` has 3 rows and `b` has 2 rows. Both have 4 columns, so the same function prints both:

```
 1  2  3  4
 5  6  7  8
 9 10 11 12

 0  0  0  0
 1  1  1  1
```

Slide 11 — Careful: both brackets empty

Try to answer before you look at the next slide: does `int g[][]` compile?

Slide 12 — No: gcc stops

It does not compile. gcc says:

```
error: array type has incomplete element type 'int[]'
```

The fix: write the number of columns, `int g[][3]`. The file is `03_bug_no_cols.c`: it fails on purpose.

Slide 13 — Why C needs the columns

In memory, a grid is one long line of boxes: row 0, then row 1 right after it, then row 2. With 3 columns, row 1 starts at box 3. With 5 columns it would start at box 5. To find `g[1][0]`, C must know how long one row is. That is the number of columns.

Slide 14 — The function fills main's grid

`times_table` fills the grid it gets: each cell is `(r + 1) * (c + 1)`. Then `main` prints it:

```
 1  2  3  4
 2  4  6  8
 3  6  9 12
```

`main` never filled `t` itself. The grid is **not copied**: the function writes straight into `main`'s grid, the same as with a 1D array in class 10. The file is `04_times_table.c`.

Slide 15 — A function that returns the total

`total` adds every cell and returns the sum. `sum = 0` comes once, before both loops. For `{{1, 2, 3}, {4, 5, 6}}` it prints `total: 21`. The file is `05_total.c`.

Slide 16 — Matrices

Part 2: matrices.

Slide 17 — A matrix is a grid of numbers

A matrix is a grid of numbers, as in maths. We call the indexes `i` (row) and `j` (column): `m[i][j]`. Two loops visit every cell, `i` outside and `j` inside. A **square** matrix has as many rows as columns.

Slide 18 — Add two matrices

Add the cells in the same place:

```
s[i][j] = a[i][j] + b[i][j];
```

`{{1, 2, 3}, {4, 5, 6}}` plus `{{10, 20, 30}, {40, 50, 60}}` gives `11 22 33` and `44 55 66`. The file is `06_matrix_add.c`.

Slide 19 — Transpose: rows become columns

The transpose turns each row into a column. The indexes swap places:

```
t[j][i] = m[i][j];
```

`m` is `1 2 3 / 4 5 6 / 7 8 9`. `t` is `1 4 7 / 2 5 8 / 3 6 9`: the first row of `m` is the first column of `t`. The file is `07_transpose.c`.

Slide 20 — Predict: two diagonals

Try to answer before you look at the next slide.

Slide 21 — Two diagonals

- **Main diagonal** (top left to bottom right): the row and the column are equal, `m[i][i]`. Here 2 + 5 + 8 = **15**.
- **Other diagonal** (top right to bottom left): the column is `N - 1 - i`, so `m[i][N - 1 - i]`. Here 1 + 5 + 6 = **12**.

One loop is enough: on a diagonal the column follows from the row. The file is `08_diagonals.c`.

Slide 22 — Which cells?

For `i` = 0, 1, 2 the main diagonal uses `m[0][0]`, `m[1][1]`, `m[2][2]`, and the other diagonal uses `m[0][2]`, `m[1][1]`, `m[2][0]`. The two diagonals share the middle cell.

Slide 23 — Careful: transpose in place

Try to answer before you look at the next slide.

Slide 24 — Nothing changed

The matrix is printed **unchanged**: `1 2 3 / 4 5 6 / 7 8 9`. gcc gives no warning; the mistake is in the logic.

Why: when `i` is 0 and `j` is 1, the program swaps 2 and 4. Later, when `i` is 1 and `j` is 0, it swaps the same two cells again, back to where they were. Every pair is swapped twice. The file is `09_bug_transpose_twice.c`.

Slide 25 — Swap each pair once

Start the inner loop at `j = i + 1`:

```
for (int j = i + 1; j < N; j++)
```

Now only the cells above the diagonal are visited, and each pair is swapped once. The output is the transpose, `1 4 7 / 2 5 8 / 3 6 9`. The file is `10_transpose_in_place.c`.

Slide 26 — A table

Part 3: a table of data.

Slide 27 — Points: 4 players, 3 rounds

Each row is a player, each column is a round: `int pts[PLAYERS][ROUNDS]`. Player 1 scored 7, 9 and 4. There are two kinds of questions: one answer per player (a row), or one answer per round (a column).

Slide 28 — Per row: each player's average

One answer per player, so the **player loop is outside**. For each player, `sum` starts at 0, the inner loop adds the rounds, and `(double) sum / ROUNDS` is the average. `(double)` makes the division exact: 20 / 3 alone would be 6.

```
player 1: 6.67
player 2: 6.33
player 3: 6.00
player 4: 7.33
```

The file is `11_points_table.c`.

Slide 29 — Per column: the best of each round

One answer per round, so the **round loop is outside**. `best` starts at the first player, `pts[0][r]`, and the inner loop goes down the players: the maximum from class 10, once for each column.

```
round 1 best: 9
round 2 best: 9
round 3 best: 10
```

Slide 30 — Which loop goes outside?

The outside loop is the thing you want **one answer for**.

- One answer per row: the row loop outside.
- One answer per column: the column loop outside.

The index order never changes: `pts[p][r]`, the row first, always.

Slide 31 — The cells around a cell

Part 4: the cells around a cell. **Project 2 needs this part.**

Slide 32 — The 8 neighbours of (r, c)

The neighbours of a cell `(r, c)` are the cells that touch it: above, below, left, right, and the four diagonal ones. Their rows are `r - 1`, `r`, `r + 1`, and their columns are `c - 1`, `c`, `c + 1`. The cell itself is in the middle and is **not** its own neighbour.

Slide 33 — Two small loops do it

Two small loops give all the positions:

```
for (int dr = -1; dr <= 1; dr++) {
    for (int dc = -1; dc <= 1; dc++) {
        // the position (r + dr, c + dc)
    }
}
```

That is 3 × 3 = 9 positions. When `dr == 0 && dc == 0` it is the cell itself: skip it, and 8 neighbours are left.

Slide 34 — Question: how many neighbours?

Try to answer before you look at the next slide. Draw the grid if it helps.

Slide 35 — Question: how many neighbours?

- A **corner** cell has **3** neighbours.
- An **edge** cell has **5**.
- A **middle** cell has **8**.

The other positions would be off the grid: row -1 or column -1 does not exist.

Slide 36 — Careful: past the right edge

Try to answer before you look at the next slide. The grid has columns 0 to 3, and the program reads column 4.

Slide 37 — A wrong neighbour, and no error

The program prints `cell: 4` and `right: 5`. No error, and no warning, even with `-Wall`.

There is no column 4. In memory, the box after `g[0][3]` is the first box of row 1, `g[1][0]`, which holds 5. C does not stop you: it reads the wrong cell. **You** must check that a position is on the grid before you read it. The file is `13_bug_right_edge.c`.

Slide 38 — is_inside: on the grid or not?

```
int is_inside(int r, int c) {
    return r >= 0 && r < ROWS && c >= 0 && c < COLS;
}
```

It returns 1 when the row is from 0 to `ROWS - 1` and the column from 0 to `COLS - 1`, and 0 otherwise. With 4 rows and 5 columns: `is_inside(0, 0)` is 1, `is_inside(3, 4)` is 1, `is_inside(-1, 2)` is 0, `is_inside(2, 5)` is 0. The file is `12_is_inside.c`.

Slide 39 — Count the neighbours of a cell

`count_neighbours(r, c)` uses the two loops over `dr` and `dc`. If both are 0, `continue` skips the cell itself. Otherwise, if `(r + dr, c + dc)` is on the grid, it counts it. The file is `14_count_neighbours.c`.

Slide 40 — Every cell of a 4 by 5 grid

`main` prints the answer for every cell:

```
3 5 5 5 3
5 8 8 8 5
5 8 8 8 5
3 5 5 5 3
```

3 in the corners, 5 on the edges, 8 in the middle.

Slide 41 — A seating chart: 1 is taken

A small room with 4 rows of 5 seats. 1 means taken, 0 means free. `seats[2][1]` is 1: row 2, seat 1 is taken. The question: how many taken seats are around a seat?

Slide 42 — Inside taken_around(s, r, c)

`taken_around` gets the chart `s` and a seat `(r, c)`. The same two loops. `nr` and `nc` are the neighbour's row and column. The seat itself is skipped. Then:

```
if (is_inside(nr, nc) && s[nr][nc] == 1) {
    count++;
}
```

A neighbour counts when it is on the chart **and** taken. The file is `15_taken_around.c`.

Slide 43 — Predict: count by hand

Try to count on the chart before you look at the next slide.

Slide 44 — All four answers

```
(0, 0): 1
(0, 2): 2
(2, 2): 4
(3, 4): 2
```

Around (2, 2) the taken seats are (1, 1), (2, 1), (2, 3) and (3, 3): 4. Around the corner (0, 0), only (1, 1) is taken: 1.

Slide 45 — Check first, then read

The order inside the condition matters:

```
is_inside(nr, nc) && s[nr][nc] == 1
```

`&&` stops at the first part that is false. If the position is off the chart, `is_inside` is 0, so `s[nr][nc]` is never read. Written the other way round, C would read outside the chart first, as on slide 37. Always check first, then read.

Slide 46 — A grid of char: each row is a string

A grid can hold characters. Here each row of the map is a string:

```
char map[ROWS][COLS + 1] = {"X..XX", ".X..X", "XX.X.", "...XX"};
```

The rows have `COLS + 1` boxes: 5 characters and the end mark `'\0'`. `map[2][1]` is one character, `'X'`.

Slide 47 — Print the map, count the X

Each cell is printed with `%c`. The row numbers are printed on the left, the column numbers on top. While printing, the program counts the `'X'` cells:

```
  01234
0 X..XX
1 .X..X
2 XX.X.
3 ...XX
taken: 10
```

The file is `16_char_map.c`.

Slide 48 — Your turn

Part 5: one function on paper.

Slide 49 — Your turn: count_value

Write `int count_value(int g[][COLS], int rows, int v)`. It returns how many cells of the grid are equal to `v`. For the grid on the slide, `count_value(a, 3, 2)` is 3. Try it before you look at the next slide.

Slide 50 — count_value: one answer

```
int count_value(int g[][COLS], int rows, int v) {
    int count = 0;
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < COLS; c++) {
            if (g[r][c] == v) {
                count++;
            }
        }
    }
    return count;
}
```

`count = 0` comes once, before the loops. For 2 it returns 3; for 10 it returns 0. The file is `17_task_count_value.c`.

Slide 51 — Today in one slide

- A grid parameter is `int g[][COLS]`, with the number of rows as a separate parameter.
- The function works on the caller's grid: it is not copied.
- Matrices: `s[i][j] = a[i][j] + b[i][j]`; transpose `t[j][i] = m[i][j]`.
- One answer per row: rows outside. One answer per column: columns outside.
- Neighbours: `dr` and `dc` from -1 to 1, skip the cell itself, and `is_inside` **before** you read.

Slide 52 — Before next week

- **Knowledge check 14** on Blackboard, about today's slides.
- **Project 2** (Treasure Hunt) is due **Monday 9 November, 23:59**. Part 4 of today is what its clues need.
- **Midterm: Friday 30 October**, classes 1 to 14.
- The next class is on **Tuesday 27 October**.
- Today's code: Blackboard, Week 7 module, *Class 14 code*.